

Rezolvare Completă și Detaliată

Concursul Național pentru Ocuparea Posturilor Didactice

Model 2026 - Informatică și Tehnologia Informației

Cuprins

SUBIECTUL I (30 de puncte)	2
1. Structura de date: Coadă	2
a) Accesul la elemente	2
b) Descriere și Exemplificare pentru o coadă cu 4 elemente	2
MODUL 1: Reprezentarea Secvențială (Vectori/Tablouri)	2
Varianta 1A: Implementare Liniară fără Deplasare (cu pointeri/indici glisanți)	2
Varianta 1B: Implementare Liniară cu Deplasarea Elementelor (Shifting)	2
Varianta 1C: Implementare Circulară (cu Aritmetică Modulară) -	
Recomandată	3
MODUL 2: Reprezentarea Înlănțuită (Noduri Dinamice)	4
Varianta 2A: Listă Simplu Înlănțuită cu Pointeri Head/Tail	4
Varianta 2B: Listă Dublu Înlănțuită	5
c) Utilizare într-o problemă: Algoritmul Lee	6
METODA A: Folosind coadă implementată manual (Vector Liniar)	6
METODA B: Folosind biblioteca standard C++ (std::queue din STL)	6
2. Arhitectura sistemelor de calcul: Unitatea Centrală de Prelucrare (UCP)	8
a) Integrare și Organizare: Von Neumann vs Harvard	8
b) Două componente fundamentale ale UCP:	8
c) Patru parametri tehnici ai microprocesorului:	8
SUBIECTUL al II-lea (30 de puncte)	8
1. Programare Backtracking: Combinări bancnote	8
METODA A: Abordare prin Backtracking Recursiv	8
METODA B: Abordare prin Backtracking Iterativ (Simulare Stivă)	10
2. Algoritmi eficienți: Suprapunere Programări Cabinet Medic	13
METODA 1: Algoritm Liniar $O(N)$ timp, $O(1)$ spațiu (pentru date sortate în fișier)	13
METODA 2: Tabel de Frecvențe $O(N)$ timp, $O(1)$ spațiu suplimentar (soluție generală pentru date nesortate)	14
SUBIECTUL al III-lea (30 de puncte)	15
1. Proiectare Test de Evaluare și Barem (Secvența A)	15
TEST DE EVALUARE (INFORMATICĂ - Clasa a X-a)	15
BAREM DE EVALUARE ȘI NOTARE	16
2. Proiectarea unei strategii didactice pentru Secvența B (Animații și Modele 3D)	17
a) Două funcții ale unui mijloc de învățământ în procesul didactic:	17
b) Strategia didactică bazată pe modelarea 3D cu Tinkercad	17
Scenariul Didactic Detaliat (Dialog și Activități)	17

SUBIECTUL I (30 de puncte)

1. Structura de date: Coadă

a) Accesul la elemente

O coadă este o structură de date liniară de tip **FIFO (First-In, First-Out)**, în care elementele sunt inserate pe la un capăt (numit **Spate / Rear**) și sunt șterse pe la celălalt capăt (numit **Față / Front**).

- Accesul direct este permis exclusiv la cele două extremități.
- Elementul aflat la poziția `front` este singurul care poate fi citit sau eliminat în mod direct.
- Adăugarea unui element nou se face exclusiv la poziția `rear`.

—

b) Descriere și Exemplificare pentru o coadă cu 4 elemente

Fie coada în care se inserează succesiv valorile A, B, C, D, apoi se realizează o ștergere (rezultând eliminarea lui A).

MODUL 1: Reprezentarea Secvențială (Vectori/Tablouri)

Definim trei moduri de implementare secvențială:

Varianta 1A: Implementare Liniară fără Deplasare (cu pointeri/indici glisanți)

- **Descriere:** Utilizăm un vector `Q` și doi indici, `front` și `rear`. La inserare, `rear` se incrementează. La ștergere, `front` se incrementează. Această variantă este simplă, dar duce la risipă de memorie (spațiul din stânga lui `front` rămâne neutilizat).
- **Etape pentru A, B, C, D:**
 1. Inițial: `front = 0`, `rear = 0`.
 2. Inserări: `Q[0]='A'`, `Q[1]='B'`, `Q[2]='C'`, `Q[3]='D'`. `rear` devine 4.
 3. Ștergere: se extrage `Q[0]` ('A'), iar `front` devine 1. Coadă activă este acum între indicii [1, 3].

• Cod C++:

```
char Q[100];
int front = 0, rear = 0;
void enqueue(char x) { Q[rear++] = x; }
char dequeue() { return Q[front++]; }
```

• Cod Pascal:

```
var
  Q: array[0..99] of char;
  front: integer = 0;
  rear: integer = 0;
procedure enqueue(x: char); begin Q[rear] := x; rear := rear + 1; end;
function dequeue: char; begin dequeue := Q[front]; front := front + 1; end;
```

Varianta 1B: Implementare Liniară cu Deplasarea Elementelor (Shifting)

- **Descriere:** La fiecare operație de ștergere, toate elementele rămase în vector sunt deplasate cu o poziție spre stânga. Indicele `front` rămâne fixat permanent la 0, iar `rear` se decrementează. Dezavantajul este complexitatea $O(N)$ la ștergere.
- **Etape:**
 1. Inserare: `Q[0]='A'`, `Q[1]='B'`, `Q[2]='C'`, `Q[3]='D'`. `rear = 4`.

2. Ștergere: se salvează $Q[0]$. Se face deplasarea: $Q[0]=Q[1]$ (B), $Q[1]=Q[2]$ (C), $Q[2]=Q[3]$ (D). rear devine 3.

• **Cod C++:**

```
char Q[100];
int rear = 0;
void enqueue(char x) { Q[rear++] = x; }
char dequeue() {
    char sters = Q[0];
    for (int i = 0; i < rear - 1; ++i) Q[i] = Q[i+1];
    rear--;
    return sters;
}
```

• **Cod Pascal:**

```
var
    Q: array[0..99] of char;
    rear: integer = 0;
procedure enqueue(x: char); begin Q[rear] := x; rear := rear + 1; end;
function dequeue: char;
var i: integer; sters: char;
begin
    sters := Q[0];
    for i := 0 to rear - 2 do Q[i] := Q[i+1];
    rear := rear - 1;
    dequeue := sters;
end;
```

Varianta 1C: Implementare Circulară (cu Aritmetică Modulară) - Recomandată

- **Descriere:** Vectorul este tratat circular folosind operatorul modulo %. Când indicii ajung la capătul vectorului, se întorc la 0. Evită risipa de spațiu și are operații în $O(1)$.

• **Cod C++:**

```
const int CAPACITATE = 5; // suficient pentru 4 elemente
char Q[CAPACITATE];
int front = 0, rear = 0, dimensiune = 0;
void enqueue(char x) {
    if (dimensiune < CAPACITATE) {
        Q[rear] = x;
        rear = (rear + 1) % CAPACITATE;
        dimensiune++;
    }
}
char dequeue() {
    if (dimensiune > 0) {
        char sters = Q[front];
        front = (front + 1) % CAPACITATE;
        dimensiune--;
        return sters;
    }
    return '\0';
}
```

• **Cod Pascal:**

```
var
    Q: array[0..4] of char;
```

```

front: integer = 0;
rear: integer = 0;
dimensiune: integer = 0;
procedure enqueue(x: char);
begin
    if dimensiune < 5 then begin
        Q[rear] := x;
        rear := (rear + 1) mod 5;
        dimensiune := dimensiune + 1;
    end;
end;
function dequeue: char;
begin
    if dimensiune > 0 then begin
        dequeue := Q[front];
        front := (front + 1) mod 5;
        dimensiune := dimensiune - 1;
    end;
end;

```

MODUL 2: Reprezentarea Înlănțuită (Noduri Dinamice)

Varianta 2A: Listă Simplu Înlănțuită cu Pointeri Head/Tail

- **Descriere:** Fiecare nod reține informația și o legătură către următorul. Menținem pointerii prim (head) pentru ștergere și ultim (tail) pentru inserare rapidă în $O(1)$.

- **Cod C++:**

```

struct Nod {
    char info;
    Nod* urm;
};
Nod *prim = nullptr, *ultim = nullptr;

void enqueue(char val) {
    Nod* nou = new Nod{val, nullptr};
    if (ultim == nullptr) {
        prim = ultim = nou;
    } else {
        ultim->urm = nou;
        ultim = nou;
    }
}

char dequeue() {
    if (prim == nullptr) return '\0';
    Nod* temp = prim;
    char val = temp->info;
    prim = prim->urm;
    if (prim == nullptr) ultim = nullptr;
    delete temp;
    return val;
}

```

- **Cod Pascal:**

```

type
    PNode = ^TNode;

```

```

Tnod = record
    info: char;
    urm: Pnod;
end;
var
    prim: Pnod = nil;
    ultim: Pnod = nil;

procedure enqueue(val: char);
var nou: Pnod;
begin
    new(nou); nou^.info := val; nou^.urm := nil;
    if ultim = nil then begin
        prim := nou; ultim := nou;
    end else begin
        ultim^.urm := nou;
        ultim := nou;
    end;
end;

function dequeue: char;
var temp: Pnod;
begin
    if prim = nil then dequeue := #0
    else begin
        temp := prim;
        dequeue := temp^.info;
        prim := prim^.urm;
        if prim = nil then ultim := nil;
        dispose(temp);
    end;
end;

```

Varianta 2B: Listă Dublu Înlanțuită

- **Descriere:** Fiecare nod reține adresa nodului anterior și a celui următor. Aceasta oferă flexibilitate maximă de navigare bidirecțională.
- **Cod C++:**

```

struct NodDublu {
    char info;
    NodDublu* ant;
    NodDublu* urm;
};
NodDublu *prim = nullptr, *ultim = nullptr;

void enqueueDublu(char val) {
    NodDublu* nou = new NodDublu{val, ultim, nullptr};
    if (ultim == nullptr) {
        prim = ultim = nou;
    } else {
        ultim->urm = nou;
        ultim = nou;
    }
}

```

c) Utilizare într-o problemă: Algoritmul Lee

- **Enunț:** Să se determine distanța minimă (numărul de pași) de la o poziție de start (x_s, y_s) la o destinație (x_f, y_f) într-o matrice binară (labirint) de dimensiuni $n \times m$, unde 0 reprezintă celulă liberă, iar 1 obstacol.

METODA A: Folosind coadă implementată manual (Vector Liniar)

Cod C++:

```
#include <iostream>
using namespace std;

struct Coordonate {
    int x, y;
};

Coordonate Q[10000];
int frontIdx = 0, rearIdx = 0;

int labirint[100][100], dist[100][100];
int n = 5, m = 5;
int dx[] = {-1, 0, 1, 0}, dy[] = {0, 1, 0, -1};

int lee(int xs, int ys, int xf, int yf) {
    dist[xs][ys] = 1;
    Q[rearIdx++] = {xs, ys};

    while (frontIdx < rearIdx) {
        Coordonate c = Q[frontIdx++];
        if (c.x == xf && c.y == yf) return dist[xf][yf] - 1;

        for (int i = 0; i < 4; ++i) {
            int nx = c.x + dx[i];
            int ny = c.y + dy[i];
            if (nx >= 1 && nx <= n && ny >= 1 && ny <= m) {
                if (labirint[nx][ny] == 0 && dist[nx][ny] == 0) {
                    dist[nx][ny] = dist[c.x][c.y] + 1;
                    Q[rearIdx++] = {nx, ny};
                }
            }
        }
    }

    return -1; // Destinație inaccesibilă
}

int main() {
    // Exemplu de labirint 5x5 complet liber
    int rez = lee(1, 1, 3, 3);
    cout << "Pași minimi: " << rez << endl;
    return 0;
}
```

METODA B: Folosind biblioteca standard C++ (std::queue din STL)

- **Descriere:** Utilizarea std::queue simplifică masiv codul și garantează performanță optimă, eliminând gestionarea manuală a indicilor de front/rear și a depășirii vectorului.

Cod C++:

```
#include <iostream>
#include <queue>
#include <vector>

using namespace std;

struct Punct {
    int x, y;
};

int leeSTL(const vector<vector<int>>& lab, Punct start, Punct stop) {
    int n = lab.size();
    int m = lab[0].size();
    vector<vector<int>> dist(n, vector<int>(m, 0));
    int dx[] = {-1, 0, 1, 0};
    int dy[] = {0, 1, 0, -1};

    queue<Punct> Q;
    Q.push(start);
    dist[start.x][start.y] = 1;

    while (!Q.empty()) {
        Punct c = Q.front();
        Q.pop();

        if (c.x == stop.x && c.y == stop.y) {
            return dist[stop.x][stop.y] - 1;
        }

        for (int i = 0; i < 4; ++i) {
            int nx = c.x + dx[i];
            int ny = c.y + dy[i];
            if (nx >= 0 && nx < n && ny >= 0 && ny < m) {
                if (lab[nx][ny] == 0 && dist[nx][ny] == 0) {
                    dist[nx][ny] = dist[c.x][c.y] + 1;
                    Q.push({nx, ny});
                }
            }
        }
    }

    return -1;
}

int main() {
    vector<vector<int>> lab = {
        {0, 0, 0, 0, 0},
        {1, 1, 0, 1, 0},
        {0, 0, 0, 0, 0},
        {0, 1, 1, 1, 1},
        {0, 0, 0, 0, 0}
    };
    int pasi = leeSTL(lab, {0, 0}, {4, 4});
    cout << "Pasi: " << pasi << endl;
    return 0;
}
```

2. Arhitectura sistemelor de calcul: Unitatea Centrală de Prelucrare (UCP)

a) Integrare și Organizare: Von Neumann vs Harvard

Unitatea Centrală de Prelucrare (UCP/CPU) este nucleul hardware care interpretează și execută instrucțiunile. Din punct de vedere structural, integrarea sa în sistem urmează două mari modele arhitecturale:

1. **Arhitectura Von Neumann:** Dispune de un singur spațiu fizic de memorie partajat atât pentru instrucțiunile programului, cât și pentru date. Conexiunea se face printr-o singură magistrală de sistem, ceea ce poate crea un blocaj de performanță (Bottleneck Von Neumann).
2. **Arhitectura Harvard:** Folosește căi de acces și memorii fizice complet separate pentru instrucțiuni (memoria de program) și date (memoria de date). Aceasta permite CPU-ului să citească simultan o instrucțiune și să efectueze operații cu date din memorie, crescând substanțial viteza.

b) Două componente fundamentale ale UCP:

1. **UCC (Unitatea de Comandă și Control):** Decodifică instrucțiunile citite din memorie, generează semnalele de control care sincronizează componentele sistemului de calcul și coordonează fluxul general de date.
2. **ALU (Unitatea Aritmetico-Logică):** Efectuează operațiile aritmetice de bază (adunare, scădere) și logice (ȘI, SAU, NEGAȚIE, comparații) asupra datelor de intrare.

c) Patru parametri tehnici ai microprocesorului:

1. **Frecvența de tact (tactul CPU):** Exprimată în GigaHertz (GHz). Indică numărul de cicluri elementare de instrucțiuni executate într-o secundă. O frecvență ridicată reduce timpul de execuție al fiecărei instrucțiuni. **Valoare de generație actuală:** 5.7 GHz (Intel Core i9-14900K în mod Boost).
2. **Numărul de nuclee (Cores):** Reprezintă unități fizice de procesare independente pe același cip. Permite prelucrarea paralelă de fire de execuție multiple (multi-threading). **Valoare actuală:** 24 nuclee (8 Performance + 16 Efficient).
3. **Memoria Cache (L3):** Memorie extrem de rapidă integrată direct pe pastila procesorului, utilizată pentru a stoca copiile datelor frecvent accesate din RAM. Reduce timpul de așteptare al CPU-ului după date. **Valoare actuală:** 96 MB (AMD Ryzen 7 7800X3D folosind tehnologia 3D V-Cache).
4. **Lățimea magistralei de date interne (Word Size):** Numărul de biți procesați simultan într-un registru intern. Un procesor de 64 biți poate procesa numere mari și poate adresa cantități uriașe de memorie RAM direct (> 4 GB). **Valoare actuală:** 64 biți (arhitectură x86-64).

SUBIECTUL al II-lea (30 de puncte)

1. Programare Backtracking: Combinări bancnote

Avem de generat toate modalitățile de a obține suma s folosind cele n cupiuri date.

METODA A: Abordare prin Backtracking Recursiv

- **Descriere:** Sortăm cupiurile crescător. Funcția recursivă `back` încearcă pentru cupiura curentă `idx` toate numerele posibile de bancnote, începând de la numărul maxim posibil `sum / c[idx]` descrescător până la 0, apelându-se apoi pentru restul de sumă la nivelul inferior.

Cod C++:

```
#include <iostream>
#include <algorithm>
using namespace std;

int n;
int c[10], b[10];

void tipar() {
    for (int i = 0; i < n; ++i) {
        if (b[i] > 0) {
            cout << b[i] << "x" << c[i] << (c[i] == 1 ? "leu " : "lei ");
        }
    }
    cout << "\n";
}

void back(int idx, int sum) {
    if (sum == 0) {
        tipar();
        return;
    }
    if (idx < 0) return;
    int max_b = sum / c[idx];
    for (int bills = max_b; bills >= 0; --bills) {
        b[idx] = bills;
        back(idx - 1, sum - bills * c[idx]);
    }
    b[idx] = 0;
}

int main() {
    int s;
    if (!(cin >> s >> n)) return 0;
    for (int i = 0; i < n; ++i) {
        cin >> c[i];
        b[i] = 0;
    }
    sort(c, c + n);
    back(n - 1, s);
    return 0;
}
```

Cod Pascal:

```
program CombinariBancnoteRecurziv;
var
    s, i, j, temp: integer;
    n: integer;
    c, b: array[0..9] of integer;

procedure tipar;
var k: integer;
begin
    for k := 0 to n - 1 do
        begin
```

```

    if b[k] > 0 then
    begin
        if c[k] = 1 then write(b[k], 'x', c[k], 'leu ')
        else write(b[k], 'x', c[k], 'lei ');
    end;
end;
writeln;
end;

procedure back(idx, sum: integer);
var bills, max_b: integer;
begin
    if sum = 0 then begin tipar; exit; end;
    if idx < 0 then exit;
    max_b := sum div c[idx];
    for bills := max_b downto 0 do
    begin
        b[idx] := bills;
        back(idx - 1, sum - bills * c[idx]);
    end;
    b[idx] := 0;
end;

begin
    read(s, n);
    for i := 0 to n - 1 do begin read(c[i]); b[i] := 0; end;
    for i := 0 to n - 2 do
        for j := i + 1 to n - 1 do
            if c[i] > c[j] then
            begin
                temp := c[i]; c[i] := c[j]; c[j] := temp;
            end;
        back(n - 1, s);
    end.

```

METODA B: Abordare prin Backtracking Iterativ (Simulare Stivă)

- **Descriere:** Înlocuim recursivitatea cu un mecanism iterativ care folosește vectori de stare pentru a ține evidența nivelului curent, a numărului curent de bancnote selectat și a sumei rămase la fiecare nivel.

Cod C++:

```

#include <iostream>
#include <algorithm>
using namespace std;

void backIterativ(int s, int n, int c[]) {
    int b[10] = {0};
    int current_bill[10];
    int sum_at_level[11];

    for (int i = 0; i < n; ++i) {
        current_bill[i] = -1;
    }

    int level = n - 1;

```

```

sum_at_level[n] = s;

while (level < n) {
    if (level < 0) {
        // S-a ajuns la capătul din stânga (toate cupiurile procesate)
        if (sum_at_level[0] == 0) {
            // Afișează soluție
            for (int i = 0; i < n; ++i) {
                if (b[i] > 0) {
                    cout << b[i] << "x" << c[i] << (c[i] == 1 ? "leu " : "lei ");
                }
            }
            cout << "\n";
        }
        level++; // Backtrack
        continue;
    }

    if (current_bill[level] == -1) {
        current_bill[level] = sum_at_level[level + 1] / c[level];
    } else {
        current_bill[level]--;
    }

    if (current_bill[level] >= 0) {
        b[level] = current_bill[level];
        sum_at_level[level] = sum_at_level[level + 1] - b[level] * c[level];
        level--;
        if (level >= 0) {
            current_bill[level] = -1; // Reset nivel următor
        }
    } else {
        b[level] = 0;
        level++; // Revenire la nivelul anterior
    }
}

}

int main() {
    int s, n;
    int c[10];
    if (!(cin >> s >> n)) return 0;
    for (int i = 0; i < n; ++i) cin >> c[i];
    sort(c, c + n);
    backIterativ(s, n, c);
    return 0;
}

```

Cod Pascal:

```

program CombinariBancnoteIterativ;
var
    s, n, i, j, temp: integer;
    c: array[0..9] of integer;

procedure backIterativ(s, n: integer);
var

```

```

b, current_bill: array[0..9] of integer;
sum_at_level: array[0..10] of integer;
level, k: integer;
begin
  for k := 0 to n - 1 do
    begin
      b[k] := 0;
      current_bill[k] := -1;
    end;
  level := n - 1;
  sum_at_level[n] := s;

  while level < n do
    begin
      if level < 0 then
        begin
          if sum_at_level[0] = 0 then
            begin
              for k := 0 to n - 1 do
                begin
                  if b[k] > 0 then
                    begin
                      if c[k] = 1 then write(b[k], 'x', c[k], 'leu ')
                      else write(b[k], 'x', c[k], 'lei ');
                    end;
                  end;
                writeln;
              end;
              level := level + 1;
              continue;
            end;

            if current_bill[level] = -1 then
              current_bill[level] := sum_at_level[level + 1] div c[level]
            else
              current_bill[level] := current_bill[level] - 1;

            if current_bill[level] >= 0 then
              begin
                b[level] := current_bill[level];
                sum_at_level[level] := sum_at_level[level + 1] - b[level] * c[level];
                level := level - 1;
                if level >= 0 then
                  current_bill[level] := -1;
                end
              else
                begin
                  b[level] := 0;
                  level := level + 1;
                end;
              end;
            end;
          end;
        end;

      begin
        read(s, n);
        for i := 0 to n - 1 do begin read(c[i]); end;
      end;
    end;
  end;

```

```

for i := 0 to n - 2 do
  for j := i + 1 to n - 1 do
    if c[i] > c[j] then
      begin
        temp := c[i]; c[i] := c[j]; c[j] := temp;
      end;
    backIterativ(s, n);
  end.
end.

```

2. Algoritmi eficienți: Suprapunere Programări Cabinet Medic

Fiecare programare este dată de ziua $\times 100 +$ ora. Intervalul codurilor este [108, 3120]. Avem de determinat câte programări se suprapun (apar de mai multe ori) și trebuie actualizate.

METODA 1: Algoritm Liniar $O(N)$ timp, $O(1)$ spațiu (pentru date sortate în fișier)

- **Justificarea eficienței:** Deoarece datele din fișier sunt gata sortate crescător, valorile egale (suprapunerile) sunt vecine în șir. Putem citi fișierul număr cu număr, păstrând doar ultima valoare citită. Dacă valoarea curentă este egală cu cea precedentă, avem o suprapunere și incrementăm contorul. Spațiul suplimentar este $O(1)$ deoarece nu stocăm numerele în memorie. Timpul este $O(N)$, realizându-se o singură parcurgere a fișierului.

Cod C++:

```

#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream fin("titu2025.txt");
    int current, val, rescheduled = 0;
    if (!(fin >> current)) return 0;
    while (fin >> val) {
        if (val == current) rescheduled++;
        else current = val;
    }
    cout << rescheduled << endl;
    return 0;
}

```

Cod Pascal:

```

program EvaluareSuprapuneriSortat;
var
    fin: text;
    current, val, rescheduled: longint;
begin
    assign(fin, 'titu2025.txt'); reset(fin);
    if eof(fin) then begin writeln(0); close(fin); halt; end;
    read(fin, current);
    rescheduled := 0;
    while not eof(fin) do
        begin
            read(fin, val);
            if val = current then rescheduled := rescheduled + 1
            else current := val;
        end;
    end;
    writeln(rescheduled);
end.

```

```
end;
close(fin);
writeln(rescheduled);
end.
```

METODA 2: Tabel de Frecvențe $O(N)$ timp, $O(1)$ spațiu suplimentar (soluție generală pentru date nesortate)

- **Justificarea eficienței:** Deoarece codurile posibile aparțin intervalului restrâns $[108, 3120]$, putem folosi un vector de frecvență alocat static de dimensiune 3200. Pentru fiecare cod citit, incrementăm poziția corespunzătoare din vectorul de frecvență. La final, adunăm pentru fiecare cod care apare de cel puțin 2 ori valoarea frecvența - 1. Această abordare este extrem de robustă deoarece funcționează chiar dacă datele din fișier nu ar fi sortate.

Cod C++:

```
#include <iostream>
#include <fstream>
#include <vector>
using namespace std;

int freqv[3201]; // inițializat automat cu 0

int main() {
    ifstream fin("titu2025.txt");
    int val;
    while (fin >> val) {
        if (val >= 108 && val <= 3120) {
            freqv[val]++;
        }
    }
    fin.close();

    int rescheduled = 0;
    for (int i = 108; i <= 3120; ++i) {
        if (freqv[i] > 1) {
            rescheduled += (freqv[i] - 1);
        }
    }
    cout << rescheduled << endl;
    return 0;
}
```

Cod Pascal:

```
program EvaluareSuprapuneriiFrecventa;
var
    fin: text;
    val, i: integer;
    rescheduled: longint;
    freqv: array[108..3120] of longint;
begin
    for i := 108 to 3120 do freqv[i] := 0;

    assign(fin, 'titu2025.txt'); reset(fin);
    while not eof(fin) do
        begin
            read(fin, val);
```

```

    if (val >= 108) and (val <= 3120) then
        frecv[val] := frecv[val] + 1;
    end;
close(fin);

rescheduled := 0;
for i := 108 to 3120 do
begin
    if frecv[i] > 1 then
        rescheduled := rescheduled + (frecv[i] - 1);
    end;
    writeln(rescheduled);
end.

```

SUBIECTUL al III-lea (30 de puncte)

1. Proiectare Test de Evaluare și Barem (Secvența A)

Testul este proiectat pentru a fi administrat într-o probă scrisă de 50 de minute, vizând competențele specifice 2.1 și 2.2 din Secvența A (Subprograme).

TEST DE EVALUARE (INFORMATICĂ - Clasa a X-a)

Item 1 (15 puncte) - Alegere Multiplă Fie următorul subprogram C++:

```

void f(int &a, int b) {
    a = a + b;
    b = a - b;
}

```

Dacă variabilele globale x și y au valorile $x = 5$ și $y = 10$, care vor fi valorile lor după apelul $f(x, y)$? a) $x = 15, y = 5$

b) $x = 5, y = 10$

c) $x = 15, y = 10$

d) $x = 15, y = 15$

Item 2 (15 puncte) - Completare spații libere Completați spațiul liber astfel încât afirmația să fie corectă: „O variabilă declarată în interiorul corpului unui subprogram se numește variabilă _____ și domeniul ei de vizibilitate este limitat la _____.”

Item 3 (15 puncte) - Răspuns Scurt Precizați rolul structurii de date de tip **Stivă (Stack)** folosită de sistemul de operare la apelul subprogramei recursive.

Item 4 (20 puncte) - Întrebare Structurată Se consideră subprogramul recursiv g definit mai jos:

```

int g(int n) {
    if (n <= 0) return 0;
    return n % 10 + g(n / 10);
}

```

a) Ce valoare returnează apelul $g(4532)$? (10 puncte)

b) Explicați, pas cu pas (trace), modul de determinare a rezultatului apelului $g(4532)$ indicând stiva de apeluri. (10 puncte)

Item 5 (25 puncte) - Item Subiectiv (Rezolvare de probleme) Scrieți un subprogram complet în C++ sau Pascal numit `cifreEgale` care primește ca parametru un număr natural N ($N \in [10, 10^9]$) și returnează valoarea 1 (sau `true`) dacă toate cifrele lui N sunt egale între ele, respectiv 0 (sau `false`) în caz contrar. Exemplu: `cifreEgale(777)` returnează 1, iar `cifreEgale(787)` returnează 0.

BAREM DE EVALUARE ȘI NOTARE

- Se acordă 10 puncte din oficiu.
- Nota finală se obține prin împărțirea punctajului total (din 100) la 10.

Item 1 (15 puncte)

- Răspuns corect: **c) $x = 15$, $y = 10$** . (15p)
- **Explicație:** Parametrul a este transmis prin referință (&), deci modificările asupra lui a afectează direct variabila x (x devine $5 + 10 = 15$). Parametrul b este transmis prin valoare, deci modificările asupra lui b nu afectează variabila y (y rămâne 10).

Item 2 (15 puncte)

- Pentru completarea corectă a primului spațiu („locală”): (7.5p)
- Pentru completarea corectă a celui de-al doilea spațiu („subprogramul respectiv” / „corpul subprogramului”): (7.5p)

Item 3 (15 puncte)

- Se acordă punctaj maxim pentru explicarea rolului stivei: salvarea adreselor de revenire din funcții, stocarea parametrilor transmiși la apel și a variabilelor locale specifice fiecărui nivel de apel recursiv. (15p)

Item 4 (20 puncte)

- a) Valoarea returnată: **14** (reprezintă suma cifrelor lui 4532). (10p)
- b) Etapele de trasare în stivă (10p):
 - `g(4532)` returnează $2 + g(453)$
 - `g(453)` returnează $3 + g(45)$
 - `g(45)` returnează $5 + g(4)$
 - `g(4)` returnează $4 + g(0)$
 - `g(0)` atinge cazul de bază și returnează 0.
 - Reconstituirea sumei pe stivă: $0 \rightarrow 4 + 0 = 4 \rightarrow 5 + 4 = 9 \rightarrow 3 + 9 = 12 \rightarrow 2 + 12 = 14$.

Item 5 (25 puncte)

- Declararea corectă a antetului funcției și a tipului returnat: (5p)
- Extragerea corectă a ultimei cifre ca cifră de referință: (5p)
- Structură repetitivă corectă pentru compararea tuturor celorlalte cifre: (10p)
- Returnarea valorii corecte (1/`true` sau 0/`false`): (5p)

Cod de referință (C++):

```
bool cifreEgale(int N) {
    int ref = N % 10;
    N /= 10;
    while (N > 0) {
        if (N % 10 != ref) return false;
        N /= 10;
    }
}
```

```
    return true;
}
```

2. Proiectarea unei strategii didactice pentru Secvența B (Animații și Modele 3D)

a) Două funcții ale unui mijloc de învățământ în procesul didactic:

1. **Funcția cognitivă (de informare):** Mijlocul de învățământ servește drept sursă directă de cunoaștere, facilitând înțelegerea conceptelor abstracte prin reprezentări senzoriale, vizuale și grafice dinamice (ex. elevul observă rotația direct pe ecran).
2. **Funcția motivatoare (simulativă și estetică):** Stimulează interesul elevilor pentru studiu, le captează atenția și trezește dorința de investigație prin interactivitate directă.

b) Strategia didactică bazată pe modelarea 3D cu Tinkercad

- **Mijloc de învățământ:** Aplicația web gratuită de modelare CAD 3D **Autodesk Tinkercad**.
- **Metoda didactică:** **Învățarea prin descoperire** combinată cu **Exercițiul**.
- **Forma de organizare:** Activitate individuală pe calculator.
- **Activitate de învățare:** Elevii vor construi modelul 3D al unei căsuțe prin gruparea și editarea proprietăților geometrice (cub pentru corp, prismă pentru acoperiș), utilizând operațiile de dimensionare, poziționare și transparență.

Scenariul Didactic Detaliat (Dialog și Activități)

1. **Constituirea cadrului de învățare (Captarea atenției) - 5 min:**
 - **Profesorul:** „Astăzi vom fi arhitecți digitali! Vom construi o căsuță 3D direct în browser folosind forme geometrice elementare.” Solicită elevilor să acceseze conturile de Tinkercad.
 - **Elevii:** Pornesc calculatoarele, deschid Tinkercad și creează un proiect nou de modelare 3D.
2. **Faza de Explorare și Descoperire Dirijată - 25 min:**
 - **Profesorul:** „În partea dreaptă aveți panoul cu forme de bază. Trageți pe planul de lucru (Workplane) un corp roșu de tip 'Box' (cub). Cum îi putem modifica dimensiunile pentru a obține un perete?”
 - **Elevii:** Observă punctele albe și negre (ancorele de dimensionare) din jurul cubului. Trăgând de ele, modifică lungimea, lățimea și înălțimea cubului.
 - **Profesorul:** „Excelent! Acum avem pereții. Avem nevoie de un acoperiș. Trageți o formă de tip 'Roof' (prismă triunghiulară). Încercați să o așezați deasupra casei.”
 - **Elevii:** Încearcă să poziționeze acoperișul, dar acesta intră în interiorul cubului. Profesorul intervine întrebând cum putem ridica un obiect pe axa verticală Z.
 - **Profesorul:** „Observați conul negru situat deasupra acoperișului. Faceți click pe el și trageți în sus. Ce se întâmplă?”
 - **Elevii:** „Obiectul se ridică deasupra solului! Îl ridicăm exact la înălțimea cubului.”
 - **Profesorul:** „Corect. Acum, pentru a tăia o ușă sau o fereastră în perete, vom folosi o formă cu proprietatea 'Hole' (transparență/decupaj). Trageți un cub mai mic, bifați opțiunea 'Hole', introduceți-l parțial în perete și apoi selectați ambele forme și apăsați butonul 'Group' (Ctrl+G). Ce observați?”

- **Elevii:** „A apărut o deschidere în perete! Geometria transparentă a decupat materialul cubului.”

3. Consolidare (Exercițiu individual) - 15 min:

- **Profesorul:** Adresează o provocare suplimentară: adăugarea unui coș de fum utilizând un cilindru rotit la 15 grade.
- **Elevii:** Aplică operația de rotire folosind săgețile curbe de pe cilindru și assemblează coșul pe acoperiș.

4. Evaluare și feed-back - 5 min:

- **Profesorul:** Face o trecere în revistă a proiectelor pe ecranul principal. Evaluează gradul de aliniere al obiectelor și corectitudinea utilizării operațiilor de grupare.